Entrega de Inteligência Artificial

Este documento descreve como o projeto atende aos requisitos da disciplina de Inteligência Artificial do 2º ano e indica o arquivo que comprova cada um no GitHub.

Período
2026

Responsável
Equipe do projeto

Sumário

O que está nesta entrega

01
Visão geral 3
O que a disciplina pede e o que foi entregue

02
Requisitos mínimos 4
API, agentes, LangGraph, memória, MCP, A2A, RAG, juiz, guardrails e arquitetura

03
Requisitos extras 7
Complexidade do projeto

04
Entregas adicionais 8
O que o projeto reúne nesta área

05
Repositórios da disciplina 9
Papel de cada repositório na entrega

01

O projeto tem um assistente multiagente com API, memória e revisão das respostas

O que a disciplina pede e como este documento comprova a entrega?

O que a disciplina pede

A disciplina pede uma API em FastAPI para um sistema de IA baseado em modelos de linguagem, com escopo definido pelo grupo. O sistema deve ser multiagente, com ao menos cinco agentes criados com LangChain e orquestrados com LangGraph. Também pede controle de sessões por usuário, memória de longo prazo, integrações por MCP e A2A, RAG com fonte externa, um agente juiz, guardrails, observabilidade das chamadas e um desenho de arquitetura.

O que foi entregue

Nossa equipe construiu o ai-assistant, uma aplicação FastAPI que executa um grafo LangGraph com oito agentes especializados. Os modelos Gemini e Groq são acessados pelo LangChain. A sessão de cada conversa e a memória de longo prazo ficam no MongoDB, e o FAQ da plataforma é consultado por RAG em um banco vetorial Qdrant. O assistente carrega ferramentas do servidor MCP mcp-database, delega pedidos de operação ao ai-operational por A2A e também atende a delegações de outros agentes. Um agente juiz avalia as respostas, e guardrails de entrada e de saída revisam as mensagens. A telemetria de cada etapa segue para o api-messenger, e o Sandbox, interface de administração do ambiente de QA, mostra custo, latência, erros, resoluções e retorno em painéis.

Como ler este documento

Cada requisito tem um identificador (M para os mínimos e E para os extras), o enunciado da disciplina, a descrição do que foi entregue e um link para o arquivo no GitHub, na branch main. Quando o mesmo padrão aparece em mais de um repositório, o documento mostra um exemplo e cita os demais pelo nome. Os identificadores seguem o enunciado oficial da disciplina.

02

Os requisitos mínimos estão no código e na documentação do ai-assistant

Onde está a prova de cada requisito mínimo?

Id	Requisito	Como o projeto atende	Evidência
M1	API em FastAPI (ou Flask) para um sistema de IA baseado em modelos (Gemini, GPT, DeepSeek, Groq e outros).	O ai-assistant é uma aplicação FastAPI com contrato OpenAPI. Os modelos Gemini e Groq são acessados pelo LangChain, e a cadeia de modelos usada pelos agentes com ferramentas e pelo orquestrador troca de provedor automaticamente. O mesmo padrão aparece também em ai-operational, ai-geographic e ai-validation.	openapi.yaml em ai-assistant app.py em ai-assistant
M2	O grupo define o escopo do sistema.	A visão de produto define o assistente como apoio a quem busca placas solares, itens de fixação, profissionais e empresas. Ele responde dúvidas sobre a plataforma, calcula estimativas preliminares e encaminha pedidos de operação ao agente operacional.	OVERVIEW.md em ai-assistant
M3	Sistema multiagente com no mínimo 5 agentes.	O assistente tem oito agentes, cada um em sua pasta de especialistas. São o roteador, o orquestrador, o FAQ, a calculadora solar, o especialista em painéis solares, o sugestor de profissional, o sugestor de agência e o juiz. Todos são nós do mesmo grafo, e o roteador, o orquestrador e o juiz chamam o modelo diretamente. O ai-operational e o ai-geographic são assistentes próprios que se comunicam com ele.	pasta specialist em ai-assistant graph.py em ai-assistant
M4	Uso de LangChain para a criação dos agentes.	Os agentes FAQ, sugestor de profissional, sugestor de agência, calculadora solar e especialista em painéis solares são criados com a função create_agent do LangChain, por uma classe base que monta o prompt padrão e liga a cadeia de modelos Gemini e Groq, com troca de provedor.	requirements.txt em ai-assistant pasta base em ai-assistant
M5	Uso de LangGraph para a orquestração dos agentes.	O grafo é um StateGraph do LangGraph com nós para o guardrail de entrada, o resumo, o roteador, os especialistas, o orquestrador, o juiz e o guardrail de saída. Arestas condicionais decidem o caminho, e o estado compartilhado fica em um módulo próprio.	graph.py em ai-assistant routing_edges.py em ai-assistant
M6	Controle de sessões por usuário.	O runner associa cada conversa a um thread_id e confere o dono salvo no estado contra o usuário do token. A API valida o JWT pelo JWKS do api-auth antes de chamar os modelos, e o estado da sessão é gravado no MongoDB por um checkpointer do LangGraph.	runner.py em ai-assistant auth.py em ai-assistant

Id	Requisito	Como o projeto atende	Evidência
M7	Memória de longo prazo.	Os fatos de cada usuário ficam no MongoDB e são lidos no início de cada conversa e atualizados ao fim. Um nó de resumo condensa o histórico longo para manter o contexto.	user_memory_repository.py em ai-assistant summary_node.py em ai-assistant
M8	MCP e A2A, integrações entre sistemas e/ou agentes externos.	O cliente MCP do assistente carrega as ferramentas do servidor mcp-database, que consulta ofertas, fornecedores, profissões e técnicos. Por A2A, o assistente delega pedidos de operação ao ai-operational e publica um servidor com a habilidade de recomendar produtos, profissionais e empresas, que o ai-geographic usa.	client.py em ai-assistant DELEGATION_A2A.md em ai-assistant
M9	RAG, com indicação das fontes externas usadas e no mínimo uma fonte externa consumida.	A fonte é um PDF de perguntas frequentes da plataforma, indexado em um banco vetorial Qdrant. O agente FAQ consulta os trechos recuperados por uma ferramenta de busca e responde com base neles.	FAQ_v1.pdf em ai-assistant faq_reader_node.py em ai-assistant
M10	Agente de juiz para controle de alucinações.	Depois do orquestrador, o agente juiz avalia a resposta e devolve o veredito aprovado ou reprovado, com justificativa. Uma reprovação pede uma nova tentativa ao orquestrador, e, se ela persistir, uma mensagem de bloqueio substitui a resposta.	judge_node.py em ai-assistant pasta judge em ai-assistant
M11	Implementação de guardrail.	O guardrail de entrada combina padrões de injeção de prompt, palavras de dados internos, anonimização e um classificador por modelo, e recusa a mensagem quando necessário. O guardrail de saída confere a resposta antes de entregá-la. A documentação do fluxo traz o diagrama.	input_guardrail_node.py em ai-assistant GUARDRAILS.md em ai-assistant
M12	Observabilidade e SRE, com o custo estimado para cenários de 100 e 1000 usuários semanais.	O Sandbox traz uma simulação de custo por usuários semanais. Os cenários padrão são de 100 e de 1000 usuários, e outros valores podem ser acrescentados. A projeção semanal e mensal usa o custo médio por conversa e o número de conversas por usuário por semana observados no período escolhido.	cost.ts em web-sandbox CostSimulation.tsx em web-sandbox
M13	Observabilidade e SRE, com a latência entre agentes e o tempo total de resposta.	O StepTracker mede a latência de cada chamada ao modelo e de cada ferramenta, por etapa, e envia o registro ao api-messenger. Lá, cada etapa guarda o nó, a ordem, o modelo, a conversa e o campo latencyMs. No Sandbox, os painéis mostram a latência p50 e p95 das conversas, e a linha do tempo de cada resposta traz a duração e o modelo de cada nó e o tempo total do turno.	step_tracker.py em ai-assistant timeline.tsx em web-sandbox

Id	Requisito	Como o projeto atende	Evidência
M14	Observabilidade e SRE, com o índice de erros.	Cada chamada ao modelo e cada ferramenta registram o status (ok, limite de uso, tempo esgotado, erro de conexão ou erro) e a mensagem de erro, no mesmo registro de etapa enviado ao api-messenger. No Sandbox, os painéis mostram a taxa de erro do período e por assistente, e o bloco do LangSmith traz a taxa de erro das execuções.	step_tracker.py em ai-assistant stats.ts em web-sandbox
M15	Observabilidade e SRE, com o custo e o retorno (ROI).	O Sandbox calcula o retorno na simulação de 100 e 1000 usuários semanais. O retorno é o valor dos atendimentos resolvidos, medido pelo custo de um atendimento humano informado na própria tela, menos o custo da IA, dividido pelo custo da IA.	cost.ts em web-sandbox CostSimulation.tsx em web-sandbox
M16	Observabilidade e SRE, com o custo por resolução.	Os Dashboards do Sandbox mostram as resoluções e o custo por resolução. Uma resolução é uma conversa sem erro e com resposta do assistente, e o custo por resolução divide o custo das conversas pelo número de resoluções. A simulação de 100 e 1000 usuários traz o mesmo indicador por cenário.	DashboardsPage.tsx em web-sandbox stats.ts em web-sandbox
M17	Desenho de arquitetura de alto nível.	A documentação de arquitetura do ai-assistant traz diagramas com o cliente, a API, o grafo, o MongoDB, o Qdrant, o servidor MCP, o ai-operational e os demais serviços de IA.	ARCHITECTURE.md em ai-assistant

03

O extra se apoia em vários serviços de IA que se comunicam

Quais componentes sustentam a complexidade do projeto?

Id	Requisito	Como o projeto atende	Evidência
E1	Complexidade do projeto.	O projeto reúne um assistente principal com oito agentes em um grafo LangGraph, RAG sobre o FAQ, ferramentas por MCP, delegação A2A e agente juiz. Reúne também os serviços de IA ai-operational, ai-geographic, ai-validation e ai-accessibility, a biblioteca compartilhada lib-ai e o servidor mcp-database.	ARCHITECTURE.md em ai-assistant

04
Entregas adicionais

O que o projeto reúne nesta disciplina?

O projeto reúne guardrails e o protocolo A2A em lib-ai, um servidor MCP próprio em mcp-database, validação por IA de certificados NR-10 e NR-35 e de dados de empresas em ai-validation, moderação de imagem com texto alternativo em ai-accessibility, consulta ao Google Maps por MCP em ai-geographic, consulta de endereço e de dados solares de telhados em api-google e observabilidade de LLM com custo, tokens e latência por etapa, que o api-messenger registra e o web-sandbox exibe.

Link de exemplo, lib-ai na branch main.

05
Dez repositórios sustentam a entrega

Que repositório cumpre qual papel na disciplina?

Repositório	Papel na disciplina	Link
ai-assistant	Assistente principal, com a API FastAPI, o grafo, os agentes, a memória, o RAG, o juiz e os guardrails.	ai-assistant
ai-operational	Assistente de operações, que recebe a delegação por A2A e usa FastAPI e LangGraph.	ai-operational
ai-geographic	Copiloto de campo para profissionais, que consulta o assistente principal por A2A.	ai-geographic
ai-validation	Validação de documentos e de dados de empresas, com FastAPI e LangGraph.	ai-validation
ai-accessibility	Análise de imagens e sugestão de texto alternativo.	ai-accessibility
lib-ai	Biblioteca compartilhada com seleção de provedores, leitura de JWT e comunicação entre agentes.	lib-ai
mcp-database	Servidor MCP com ferramentas de consulta de ofertas, fornecedores, profissões e técnicos.	mcp-database
api-messenger	Recebe o histórico das conversas e os registros de execução de cada etapa.	api-messenger
web-sandbox	Interface de administração do QA com simulação de custo, latência, erros e linha do tempo de cada resposta.	web-sandbox
api-google	Endereço, fuso horário, tradução e dados solares de telhados, usados pela plataforma e pelo copiloto de campo.	api-google



Entrega acadêmica.